An Edge-Enabled Wireless Split Learning Testbed: Design and Implementation

Zhe Wang, Luca Boccardo, and Yansha Deng, *Senior Member, IEEE*

***Abstract*— With advancements in the sixth generation (6G) research, the future communication network is characterized by its integration of artificial intelligence (AI). Among all AI frameworks, split learning (SL) partitions the AI model into device-side and server-side components, distributing them at both wireless device and edge server. Depending on the number of split layers, there exist a trade-off between the computation and communication costs. Until now, the impact of the wireless channel attenuation on the SL performance and impact of SL on the communication cost have been rarely studied, not to mention capturing these impacts in a practical testbed. In this letter, we build the first edge-enabled wireless split learning testbed (EWSLT), where the Intel smart edge open platform is integrated with the software-defined radio (SDR)-based 5G OpenAirInterface (OAI) platform, with split learning implemented between 5G-edge server and 5G user equipment (UE). We then evaluate the trade-off between 5G UE's computation resource and wireless communication costs by the wireless SL's effectiveness in terms of training accuracy and overall communication delay across various split layers.**

***Index Terms*— Smart-edge, split learning, OpenAirInterface, 5G wireless communication.**

I. Introduction

RECENT research on the 6G white paper highlights the potential benefits in user experience of integrating AI with upcoming 6G researches advancements [1]. This integration facilitates unique and specialized use cases by harnessing data collection, local or distributed computation offloading, and distributed training and inference of AI models across various intelligent nodes [2]. Recent research suggests that AI frameworks, such as federated learning (FL) and split learning (SL), could enhance AI task performance while safeguarding privacy [3]. Compared to FL, which has been extensively researched in both wireless and wired environments [4], the integration between SL and wireless communication has been rarely studied in current studies.

SL, as a collaborative learning framework that enables servers to efficiently handle most of the training workload, is featured by its ability in bolstering data privacy and showcasing its superiority by facilitating tasks across diverse user equipment (UE) with different computing abilities [5]. Recent research on SL has primarily focused on refining the algorithm to boost performance. For example, in [6], researchers proposed the hybrid split and federated learning (HSFL) algorithm to combine the parallel model training capabilities of FL with the model splitting methodology of SL, and it offers superior learning accuracy compared to FL and incurs a lower communication overhead than SL. In [7], a custom 6G architecture was proposed to support edge SL, which implemented resource-efficient learning frameworks and resource management strategies under a unified edge server. Although previous SL-based studies [8] have focused on model partitioning to reduce communication costs and privacy leakage [9], the prevalent use of closed-source communication platforms in edge computing and wireless communication domains has prevented these researches from implementing a standardized SL platform. This limitation hinders the evaluation of wireless communication's impact on various split layers. Recently, several robust, open-source platforms for wireless communication and edge computing have emerged, such as Intel smart-edge open, software-defined radio (SDR), etc. These platforms, with their standard APIs, can be adapted to establish a standardized platform for wireless split learning, where the edge platform may potentially enhance SL performance.

Intel smart-edge open platform[1] was recently proposed to serve as a platform for building solutions that combine cloud-native technologies to deliver AI services optimized for performance at the edge. This can be used as the open-source edge platform to run SL. The SDR facilitates the implementation of communication system components via software. Specifically, an SDR system can support multiple modulation formats and can switch between different frequency bands and standards, allowing it to cater for 5G and beyond 5G (B5G) communication research [10]. Among the various SDR projects, OpenAirInterface (OAI) stands out as an open-source platform [11], that can be used to facilitate the integration of AI algorithms with 5G protocol stacks. Until now, the impact of wireless channel attenuation on SL performance and SL on communication cost have rarely been studied, let alone capturing these impacts in a practical wireless testbed.

To fill the gap, we develop the first edge-enabled wireless split learning testbed (EWSLT), where the Intel smart edge open platform is integrated with the SDR-based 5G OAI platform, with split learning implemented between 5G-edge server and 5G UE. This letter offers two primary contributions: 1) To the best of our knowledge, this is the first letter to build the cellular-connected edge-enabled wireless split learning testbed using the SDR-based 5G OAI platform; 2) Our testbed

Manuscript received 29 February 2024; revised 14 April 2024; accepted 14 April 2024. Date of publication 18 April 2024; date of current version 12 June 2024. This work was supported in part by UKRI under the UK government's Horizon Europe funding guarantee (grant numbers 10061781 and 10087666), as part of the European Commission-funded collaborative project VERGE and MYRTUS, under the SNS JU program (grant numbers 101096034 and 101135183). This work is also a contribution by Project REASON, a UK Government funded project under the FONRC sponsored by the DSIT. The associate editor coordinating the review of this letter and approving it for publication was M. Vaezi. *(Corresponding author: Yansha Deng.)*

The authors are with the Department of Engineering, King's College London, WC2R 2LS London, U.K. (e-mail: tylor.wang@kcl.ac.uk; luca.boccardo@kcl.ac.uk; yansha.deng@kcl.ac.uk).

Digital Object Identifier 10.1109/LCOMM.2024.3390843

[1]Intel smart-edge: https://intelsmartedge.github.io/

1558-2558 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on July 09,2025 at 17:41:24 UTC from IEEE Xplore. Restrictions apply.

implements the SL framework, which integrates machine learning algorithms across various split layers, to evaluate the impact of wireless communication on the SL framework; 3) To explore the effectiveness of SL task over wireless channels, we consider semantic communication for extended reality (XR) video-based avatar skeleton extraction task, which serves as a use case to evaluate the impact of wireless communication on SL.

The rest of this work is organized as follows. Section II presents the system overview and problem formation. Section III provides the testbed hardware setup and communication in the training process. Section IV presents the experiments evaluation. Finally, Section V concludes the letter.

II. System Overview

In this letter, we first introduce the semantic information extraction task based on [12]. Then, we detail the SL training process and explain the role of wireless communication.

A. Semantic Information Extraction Task

For the specific semantic information extraction task, a point cloud $\mathbf{P}_{\text{pc}}$ is inputted into the device-side for each frame to extract data features. The input data can be expressed as

$$\mathbf{P}_{\text{pc}} = [\vec{v}_i]^{\text{T}} = [\vec{v}_1, \vec{v}_2, \cdots, \vec{v}_{N_{\text{pc}}}]^{\text{T}}, \tag{1}$$

where N_{pc} is the total number of input point cloud data points, and $\vec{v}_i$ represents a single point, the information for each point $\vec{v}_i$ can be represented as

$$\vec{v}_i = (\vec{l}_i, \vec{c}_i) = (l_{\text{x}}, l_{\text{y}}, l_{\text{z}}, c_{\text{r}}, c_{\text{g}}, c_{\text{b}}), \tag{2}$$

where the $\vec{l}_i$ and $\vec{c}_i$ represent the three-dimensional location and RGB of a point, respectively. The label of the semantic information extraction neural network, denoted as $\mathbf{D}$, represents all the skeleton information $\vec{I}_i$ of the avatar model generated through a semantic information extraction process from the downsampled point cloud $\mathbf{P}_{\text{pc}}$, which can be expressed as

$$\mathbf{D} = [\vec{I}_1, \vec{I}_2, \cdots, \vec{I}_{N_{\text{a}}}]^{\text{T}}, \tag{3}$$

where skeleton information $\vec{I}_i$ represents the geometric positions of the N_a avatars' skeletons. This equation represents the semantic extraction process, which maps the point cloud data $\mathbf{P}_{\text{pc}}$ to a more meaningful semantic representation $\mathbf{D}$ for further processing in the XR application, the whole semantic information extraction task $\mathscr{F}(\cdot)$ can be expressed as

$$\mathbf{O} = [\vec{I}'_1, \vec{I}'_2, \cdots, \vec{I}'_{N_{\text{a}}}]^{\text{T}} = \mathscr{F}\left(\mathbf{P}_{\text{pc}}, \omega\right), \tag{4}$$

where $\vec{I}'_{\text{i}}$ represents the geometric positions of predicted avatar skeleton, and the ω represents the neural network parameters of the whole semantic information extraction task. The loss function for the whole SL process could be expressed as

$$\begin{aligned}\mathscr{L} &: \min_{\{\omega\}} \lim_{T\to+\infty} \frac{1}{T}\sum_{t=0}^{T} \left|(\mathbf{O}_i - \mathbf{D}_i)\right|, \quad i \in [1, N_a] \\ &= \frac{1}{T}\sum_{t=0}^{T}\sum_{i=0}^{N_a} \sqrt{(x_i - x'_i)^2 + (y_i - y'_i)^2 + (z_i - z'_i)^2}, \end{aligned} \tag{5}$$

where T represents the total training time steps of the semantic information extraction neural network based on SL framework, (x_i, y_i, z_i) and (x'_i, y'_i, z'_i) denote the geometry label of the skeleton and the inference output of the neural network, respectively. The loss function in Eq. (5) aims to minimize the geometric difference between the semantic information label and the neural network ω inference output.

B. Split Learning

The SL framework first split the Pointconv-based skeleton extraction neural network ω into two parts: ω_{C} and ω_{E}. These two parts are divided based on four distinct layers and distributed between CPU-based UE device and the corresponding edge server, which equipped with powerful GPUs and the Intel smart edge open platform. Then, the device connects to the edge server for synchronizing training hyperparameters, such as batch size and learning rate, based on the TCP protocol [13]. For the i-th inputted data, the UE processes forward propagation with neural network ω_{C} with the l layers and then sends tensor output $\mathbf{F}_i^l$ of the l-th neural network layer to the edge server, the training process can be represented as

$$\mathbf{F}_i^l = \mathscr{F}_{\text{c}}\left(\mathbf{P}_{\text{pc}}, \omega_{\text{C}}\right), \tag{6}$$

where the process $\mathscr{F}_c(\cdot)$ denotes the forward process of the device-side neural network ω_{C}, taking the input point cloud $\mathbf{P}_{\text{pc}}$. Once the server receives the tensor $\mathbf{F}_i^l$ from the UE, it undertakes the continuation of the forward propagation at the edge server. The edge server then calculates the loss $\mathscr{L}$ between the model outputs and the semantic information labels given in Eq. (5). Subsequently, gradients derived from loss $\mathscr{L}$ are backpropagated from the final layer to the split layer. These gradients, represented as $\frac{\mathscr{L}}{\partial \mathbf{F}_i^l}$, are then sent to the UE device to facilitate the continuation of the backpropagation locally. Finally, the device receives $\frac{\mathscr{L}}{\partial \mathbf{F}_i^l}$ from the edge server and resumes backpropagation, starting from the l-th layer and proceeding to the initial hidden layer on the UE device.

C. Evaluation Metrics

1) Overall Delay: The overall inference delay T_i of the i-th batch is defined as the processing time from input data to the extraction of semantic information, which is represented as

$$T_i = T_{\text{C}} + T_{\text{F}} + T_{\text{E}} \tag{7}$$

where T_{C} represents the CPU-based UE device computing time, T_{F} represents the uplink wireless communication time, and T_{E} represents the GPU-based edge computing time. The value T_i represents the time delay of the task processing time, which represents the time of the whole model forward process.

2) Prediction Accuracy: For the predicted semantic keypoints, the prediction accuracy is used to evaluate the Average Precision (AP) overall [14], which can be expressed as

$$\text{AP} = \frac{N_{\text{T}}}{N_{\text{T}} + N_{\text{F}}} \tag{8}$$

where N_{T} and N_{F} represent the number of true and false predicted semantic label separately.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on July 09,2025 at 17:41:24 UTC from IEEE Xplore. Restrictions apply.

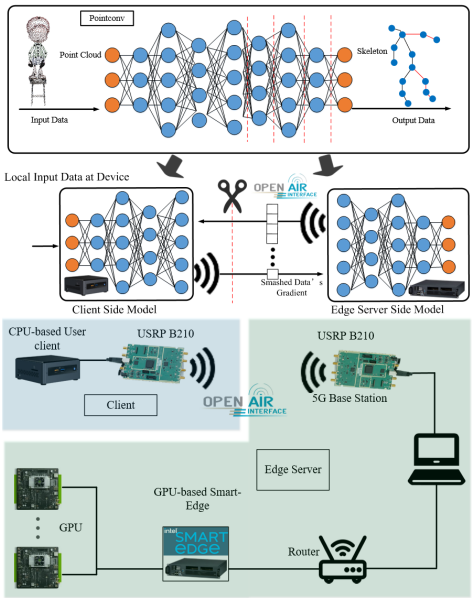


Fig. 1. Split learning based skeleton extraction task.

III. Testbed Setup

In this section, we detail the setup for our proposed testbed, as depicted in Fig.1. Specifically, we will first introduce the hardware components, which include the CPU-based UE device, the GPU-based smart-edge open platform, and their specific hardware requirements. Subsequently, we will present the edge-enabled wireless split learning testbed training process, encompassing model training and wireless communication.

A. Hardware

1) CPU-Based UE Device: The Intel next unit of computing (NUC) has been selected as the UE device. It is used to run the neural network ω_{C} at the device side and operates on a system embedded with an i5-10500 CPU and 16GB memory, running Ubuntu 20.04. One of the reasons for this choice is the NUC's support for additional expansion bays, which facilitate payload customization. The UE's components include:

- Universal software radio peripheral (USRP) [15]: To facilitate the neural network communication between the UE device and edge server, the USRP B210 is installed and connected to the Intel NUC for radio transmission and reception in our testbed. It can provide a fully integrated, single-board platform with continuous frequency coverage from 70 MHz - 6 GHz.
- Antenna: Two omnidirectional antennas with 3dB gain are installed on the TX/RX ports of USRP B210.

2) Edge Server Setup: This edge server includes a series of equipment as presented in Fig. 1, centered around a high-performance GPU-based desktop, powered by an Intel Core i7-10700 processor and an Nvidia GeForce RTX 3070 graphics card. The edge server is installed with Intel smart edge open platform to run the rest of split layers, and it is connected with a standard network interface card (NIC) and USRP with 5G OAI protocol stack for data communication.

Algorithm 1 Neural Network Training Process

1: Initialization: device side neural network ω_{C}, edge side neural network ω_{E};
2: **for** each epoch i in max epoch **do**
3: Forward device side neural network ω_{C} with input point cloud data and generate output tensor $\mathbf{F}_i^l$: $\mathbf{F}_i^l = \mathscr{F}_{\mathrm{c}}\left(\mathbf{P}_{\mathrm{pc}}, \omega_{\mathrm{C}}\right)$
4: Transmit the forward tensor to the edge server through the OAI and router with time delay T_{F}.
5: **if** $T_{\mathrm{F}} > \lambda_{\mathrm{T}}$ **then**
6: Continue
7: **end if**
8: Calculate the loss L_ω, the smashed gradient data B_ω at the edge server and update neural network ω_{E} at the edge side. $\omega_{\mathrm{E}}^{i+1} \leftarrow \omega_{\mathrm{E}}^{i} - \eta \nabla \ell\left(\omega_{\mathrm{E}}^{i}; B_{\omega_{\mathrm{E}}}\right)$
9: Transmit the gradient data $B_{\omega_{\mathrm{C}}}$ to the device side with time delay T_{B}
10: **if** $T_{\mathrm{B}} > \lambda_{\mathrm{T}}$ **then**
11: Continue
12: **end if**
13: Update the device side neural network.
14: $\omega_{\mathrm{C}}^{i+1} \leftarrow \omega_{\mathrm{C}}^{i} - \eta \nabla \ell\left(\omega_{\mathrm{C}}^{i}; B_{\omega_{\mathrm{C}}}\right)$
15: **end for**
Output: Neural network $\omega = \{\omega_{\mathrm{C}}, \omega_{\mathrm{E}}\}$

Together, these components form the edge server with SL framework, enabling the reception of forwarded backpropagation neural network parameter outputs and subsequently returning the loss and gradient values. The USRP and antenna are the same as those equipped at the device side. Other components include:

- Intel smart edge open platform: Intel's smart edge open platform leverages public APIs and Helm charts to enhance data processing efficiency. By minimizing data traffic to and from the network's edge, it could help to reduce both the network and computing burden of SL.
- NIC: The NIC port serves as the a physical interface to send and receive of data over the OAI and edge server.
- OpenAirInterface: OAI implements 3GPP technology to support 5G/B5G NewRadio, which is implemented at both the device and edge server side to achieve 5G wireless communication and AI integration.

B. Communication

For wireless communications, the detailed settings for the 5G wireless transmission configuration are presented in Table I. This table includes information on resource blocks, the distance between two USRP devices, and other relevant parameters. The complete training and communication process of the testbed is illustrated in **Algorithm 1**. This process encompasses the forward communication time, denoted as T_{F}, which involves transmitting the forward tensor output from the device to the edge server via SDR. Similarly, the back-propagation time, referred to as T_{B}, involves transmitting the loss and gradient data from the edge server to the UE device

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on July 09,2025 at 17:41:24 UTC from IEEE Xplore. Restrictions apply.

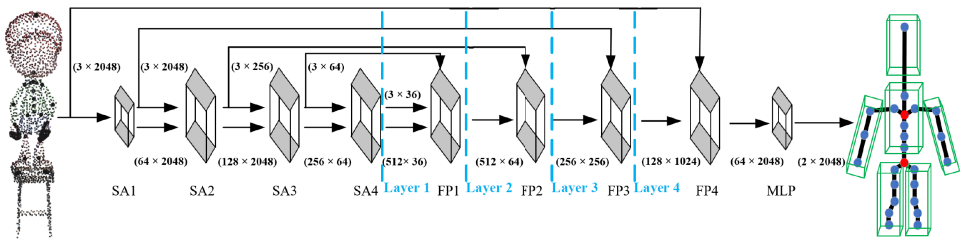


Fig. 2. Neural network architecture.

TABLE I
PLATFORM PARAMETERS

Cutting Layer	Output Tensor Size	Device Flops (MFlops)
Layer 1 (FP1)	(512,64) / 32,768	22.01
Layer 2 (FP2)	(256,256) / 65,536	31.09
Layer 3 (FP3)	(128,1024) / 131,072	49.26
Layer 4 (FP4)	(64,2048) / 131,072	58.68
No Cut	(2,2048) / 4,096	62.32

Description	Parameter	Value
Resource Blocks	rb	106
Distance	dl_distance ul_distance	1 m
Bandwidth	dl_FrequencyBand ul_FrequencyBand	band 78 band 78
Number of Antennas	nb_tx,nb_rx	2,2
Gain	max_rxgain	108 dB
Carrier Frequency	carrier_frequency	3.5 GHz
Transmission Protocol	protocol	TCP/IP
Average Packet Loss	p_loss	3.02%

via SDR. In the training process of a neural network, a large data packet will lead to increased transmission delays in an environment with packet loss. The data sizes for both forward and backpropagation are roughly similar. Therefore, considering a batch size of 16, we have set the threshold for both forward T_{F} and backpropagation T_{B} transmission times in the SL training process at 200ms to ensure that after model convergence the inference refresh rate above 30Hz (33ms/frame). As for TCP transmission, while there is no universally accepted threshold for intolerable packet loss, continuous packet loss can result in unacceptable delays. Given that the data involved in backpropagation and forward transmission in split learning are generally the same, we have established the same time threshold T_{F} to ensure a satisfactory training process. It's important to clarify that threshold is based on a testbed scenario, detailed in Table I, under wireless transmission conditions with an average packet loss rate of 3.02%. This threshold aims to investigate the impact of different split layers on the training process under varying packet loss rates, with the goal of ensuring a satisfactory training experience and assessing potential implications for XR applications. If the time delay exceeds λ_{T}, the training will start the next training epoch.

IV. EXPERIMENTS

In this section, we conduct experiments on the EWSLT testbed to evaluate the throughput, semantic information inference accuracy, and overall delay of the SL-based semantic extraction task. These evaluations are carried out for different split layers. Although our implementation uses a PointConv backbone semantic information extraction network as an example and split it into four different layers, the practical implementations designed for this EWSLT testbed can easily be adapted for various AI tasks with various split layers. TABLE I details the tensor output along with the computing flops on the UE device side for different split layers. The architecture of the semantic extraction neural network is depicted in Fig. 2, where the input for this neural network is the point cloud data in Eq. (1), while the output is detailed in Eq. (3).

Fig. 3 (a) plots the inference accuracy of SL-based semantic information extraction. It is noted that different split points result in varying numbers of neural parameters required for both forward and backward propagation communication. Errors introduced by the wireless channel can lead to retransmission in the TCP protocol, which in turn can affect the training process's performance and convergence time. After 100 epochs, the training accuracy results for different splitting layers are ranked in the following order: Layer 3 $>$ Layer 1 $>$ Layer 2 $>$ Layer 4. More specifically, the neural network with the 3-rd split layer achieves the best accuracy performance during the training process. This suggests that splitting more layers at the device helps in maintaining effective and accuracy feature extraction of the input data before the wireless communication, which will help to increase the prediction accuracy after the neural network being trained. Conversely, more backpropagation data increases the likelihood of data retransmission, which reduces convergence time. Additionally, there is significant fluctuation in the neural network at the 4-th layer, suggesting that only the edge server was trained, and the device-side neural network was not trained due to time fixed time threshold constraints. This highlights potential challenge for the trade-off between computation and communication overhead that split learning will encounter in real-world implementations.

Figure 3 (b) illustrates the variations in data size and packet loss during the training process. The size of the uplink data packets is larger than that of the downlink data packets across all four split layers. This phenomenon is due to the sharing of labels from the UE client with the edge server in the forward process, while backpropagation only requires the transmission of gradients from the first layer of the edge server. As we tested the data under consistent conditions and the distance between the two USRPs remained unchanged, the uplink / downlink packet loss are roughly proportional to the uplink / downlink packet sizes. Interestingly, with more neural

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on July 09,2025 at 17:41:24 UTC from IEEE Xplore. Restrictions apply.

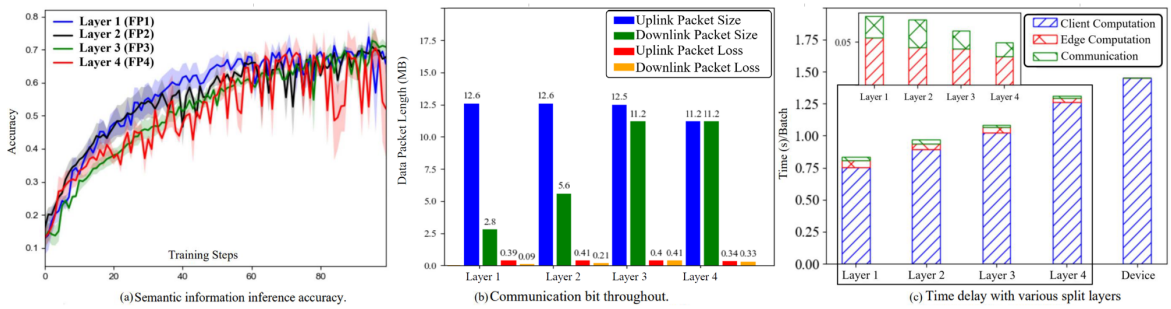


Fig. 3. Experiment results.

network layers on the UE device, the communication overhead during backpropagation also increases. The observed increase in communication overhead is primarily due to the necessity of updating parameters for each layer on the UE device. The neural network update on UE device requires the first-layer gradients from the edge server for subsequent calculations. As the splitting layer move from layer 1 to layer 4, the split layer generates more gradients at edge server. Consequently, the UE device needs to receive more gradient parameters from the edge server to enable effective backpropagation.

Fig. 3 (c) plots the overall time delay between the CPU-based UE device and the GPU-based edge server. The overall delay can be divided into three components: device computation time, wireless communication time, and edge server computation time. A lower time delay indicates a better AI task performance. The total time delay follow Layer 1 $<$ Layer 2 $<$ Layer 3 $<$ Layer 4 $<$ Device. This is because the total time delay is dominated by the local computation time, increasing the number of split layers at the device side, increases the local computation time. Additionally, with more number of split layers at the edge side, the edge computing time only constitutes a small portion of the overall time delay, indicating the edge server could significantly help to decrease the computing overhead at the device side in SL framework.

V. Conclusion

In this letter, we built the first Edge-enabled Wireless Split Learning testbed, where the Intel Smart edge open platform is integrated with the 5G OpenAirInterface platform, with split learning implemented between 5G-edge server and 5G user equipment. The testbed can support various machine learning algorithms across different split layers. We implemented a semantic information extraction task as an example to evaluate the impact of wireless communication on the split learning framework's performance, in terms of overall delay, communication cost, and inference accuracy. Our experiments yielded several insights: 1) With more number of layers at the device side, the accuracy of semantic information extraction improves due to accurate feature extraction before wireless communication. 2) In the neural network training process of SL framework, it's essential to dynamically determine the number of split layers according to the dynamic wireless communication environments to achieve a trade-off among learning accuracy, computation and communication costs. Our future work relies on both the packet loss threshold in the training process and the design of significant filters to efficiently identify and transmit only the most relevant data vectors, while reducing the transmission of statistically similar data packets.

References

[1] M. Ylianttila et al., "6G white paper: Research challenges for trust, security and privacy," 2020, *arXiv:2004.11665*.

[2] R. Liu, R. Yu-Ngok Li, M. Di Renzo, and L. Hanzo, "A vision and an evolutionary framework for 6G: Scenarios, capabilities and enablers," 2023, *arXiv:2305.13887*.

[3] Z. Yang, M. Chen, K.-K. Wong, H. V. Poor, and S. Cui, "Federated learning for 6G: Applications, challenges, and opportunities," *Engineering*, vol. 8, pp. 33–41, Jan. 2022.

[4] S. Niknam, H. S. Dhillon, and J. H. Reed, "Federated learning for healthcare domain-pipeline, applications and challenges," *IEEE Commun. Mag.*, vol. 58, no. 6, pp. 46–51, Jun. 2020.

[5] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, Feb. 2022, vol. 36, no. 8, pp. 8485–8493.

[6] X. Liu, Y. Deng, and T. Mahmoodi, "Wireless distributed learning: A new hybrid split and federated learning approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2650–2665, Apr. 2023.

[7] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split learning in 6G edge networks," 2023, *arXiv:2306.12194*.

[8] Y. Koda et al., "Communication-efficient multimodal split learning for mmWave received power prediction," *IEEE Commun. Lett.*, vol. 24, no. 6, pp. 1284–1288, Jun. 2020.

[9] S. Tuli, G. Casale, and N. Jennings, "SplitPlace: AI augmented splitting and placement of large-scale neural networks in mobile edge environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5539–5554, Sep. 2023.

[10] F. Kaltenberger, A. P. Silva, A. Gosain, L. Wang, and T.-T. Nguyen, "OpenAirInterface: Democratizing innovation in the 5G era," *Comput. Netw.*, vol. 176, Jul. 2020, Art. no. 107284.

[11] R. Mihai, R. Craciunescu, A. Martian, F. Y. Li, C. Patachia, and M.-C. Vochin, "Open-source enabled beyond 5G private mobile networks: From concept to prototype," in *Proc. 25th Int. Symp. Wireless Pers. Multimedia Commun. (WPMC)*, Oct. 2022, pp. 181–186.

[12] Z. Wang, Y. Deng, and A. H. Aghvami, "Task-oriented and semantics-aware communication framework for avatar-centric augmented reality," 2023, *arXiv:2306.15470*.

[13] S. R. Pokhrel and J. Choi, "Improving TCP performance over WiFi for Internet of Vehicles: A federated learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 6798–6802, Jun. 2020.

[14] J. Revaud, J. Almázan, R. Rezende, and C. D. Souza, "Learning with average precision: Training image retrieval with a listwise loss," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 5107–5116.

[15] N. Bello and K. O. Ogbeide, "Designing a real-time swept spectrum analyser with USRP B210," *Nigerian J. Environ. Sci. Technol.*, vol. 5, no. 2, pp. 329–339, Oct. 2021.

Authorized licensed use limited to: b-on: UNIVERSIDADE DE AVEIRO. Downloaded on July 09,2025 at 17:41:24 UTC from IEEE Xplore. Restrictions apply.